```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <title>Realm Clash - Tactical Auto-Battler</title>
    <!-- Tailwind CSS -->
    <script src="https://cdn.tailwindcss.com"></script>
    <!-- FontAwesome Icons -->
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.4.0/css/all.min.css">
    <!-- Google Fonts -->
    <link href="https://fonts.googleapis.com/css2?family=Outfit:wght@30
0;400;600;800;900&display=swap" rel="stylesheet">
    <style>
        * {
            font-family: 'Outfit', sans-serif;
            user-select: none;
        }
        body {
            background-color: #0b0f19;
            color: #e2e8f0;
            overflow: hidden;
        }
        ::-webkit-scrollbar {
            width: 6px;
            height: 6px;
```

```
        }
        ::-webkit-scrollbar-track {
            background: rgba(15, 23, 42, 0.6);
        }
        ::-webkit-scrollbar-thumb {
            background: #3b82f6;
            border-radius: 4px;
        }
        .glass-panel {
            background: rgba(15, 23, 42, 0.85);
            backdrop-filter: blur(12px);
            border: 1px solid rgba(255, 255, 255, 0.1);
        }
        .glass-panel-interactive {
            background: rgba(30, 41, 59, 0.7);
            backdrop-filter: blur(8px);
            border: 1px solid rgba(255, 255, 255, 0.08);
            transition: all 0.2s cubic-bezier(0.4, 0, 0.2, 1);
        }
        .glass-panel-interactive:hover {
            border-color: rgba(59, 130, 246, 0.5);
            transform: translateY(-2px);
            box-shadow: 0 10px 25px -5px rgba(0, 0, 0, 0.5), 0 0 15px
rgba(59, 130, 246, 0.2);
        }
        canvas {
            display: block;
            touch-action: none;
```

```
        }
        @keyframes pulse-glow {
            0%, 100% { opacity: 0.8; filter: drop-shadow(0 0 5px
currentColor); }
            50% { opacity: 1; filter: drop-shadow(0 0 15px currentColor); }
        }
        .animate-glow {
            animation: pulse-glow 2s infinite;
        }
    </style>
</head>
<body class="h-screen w-screen flex flex-col justify-between
overflow-hidden relative">

    <!-- TOP HEADER / HUD BAR -->
    <header class="w-full glass-panel z-20 px-4 py-2 flex items-center
justify-between shadow-2xl">
        <div class="flex items-center space-x-4">
            <div class="flex items-center space-x-2">
                <i class="fa-solid fa-shield-halved text-2xl text-blue-400"></i>
                <h1 class="font-black text-xl tracking-wider text-transparent
bg-clip-text bg-gradient-to-r from-blue-400 via-indigo-300
to-purple-400">REALM CLASH</h1>
            </div>
            <div class="h-6 w-px bg-slate-700"></div>
            <div class="flex items-center space-x-3">
                <span class="bg-indigo-950/80 text-indigo-300 border
border-indigo-700/50 text-xs px-2.5 py-1 rounded-full font-bold
```

```
uppercase tracking-wider flex items-center gap-1.5" id="level-badge">
                <i class="fa-solid fa-flag text-indigo-400"></i> Level <span
id="hud-level">1</span>
            </span>
            <span class="bg-slate-800/80 text-slate-300 border
border-slate-700/50 text-xs px-2.5 py-1 rounded-full font-medium hidden
sm:inline-flex items-center gap-1.5" id="scenery-badge">
                <i class="fa-solid fa-mountain-sun text-amber-400"></i>
<span id="hud-scenery">Grassy Meadow</span>
            </span>
        </div>
    </div>

    <!-- Center: Base HP Indicators -->
    <div class="hidden md:flex items-center space-x-6 w-1/3
max-w-md">
        <div class="flex-1">
            <div class="flex justify-between text-xs font-semibold mb-1">
                <span class="text-blue-400 flex items-center gap-1"><i
class="fa-solid fa-fort-awesome"></i> Your Fortress</span>
                <span id="player-hp-text"
class="text-slate-300">1000/1000</span>
            </div>
            <div class="w-full bg-slate-800/80 rounded-full h-2.5
overflow-hidden border border-slate-700/50">
                <div id="player-hp-bar" class="bg-gradient-to-r
from-blue-600 to-cyan-400 h-full w-full transition-all duration-300"></div>
            </div>
```

```
        </div>

        <div class="font-black text-xs text-slate-500">VS</div>

        <div class="flex-1">
            <div class="flex justify-between text-xs font-semibold mb-1">
                <span id="enemy-hp-text"
class="text-slate-300">1000/1000</span>
                <span class="text-red-400 flex items-center gap-1">Enemy
Citadel <i class="fa-solid fa-dungeon"></i></span>
            </div>
            <div class="w-full bg-slate-800/80 rounded-full h-2.5
overflow-hidden border border-slate-700/50">
                <div id="enemy-hp-bar" class="bg-gradient-to-r
from-red-500 to-rose-600 h-full w-full transition-all duration-300"></div>
            </div>
        </div>
    </div>

    <!-- Right: Economy & Shop Trigger -->
    <div class="flex items-center space-x-3">
        <div class="flex items-center space-x-1.5 bg-amber-950/40
border border-amber-600/40 px-3 py-1 rounded-lg text-amber-400
font-bold text-sm shadow-inner">
            <i class="fa-solid fa-coins text-amber-400 animate-glow"></i>
            <span id="hud-coins">100</span>
        </div>
        <div class="flex items-center space-x-1.5 bg-cyan-950/40 border
```

```
border-cyan-500/40 px-3 py-1 rounded-lg text-cyan-300 font-bold
text-sm shadow-inner">
                <i class="fa-solid fa-gem text-cyan-400 animate-glow"></i>
                <span id="hud-gems">0</span>
            </div>
            <button id="btn-open-shop" class="bg-gradient-to-r
from-purple-600 to-indigo-600 hover:from-purple-500
hover:to-indigo-500 text-white px-3 py-1.5 rounded-lg text-xs font-bold
tracking-wide transition shadow-lg hover:shadow-purple-500/25 flex
items-center gap-1.5">
                <i class="fa-solid fa-store"></i> <span class="hidden
sm:inline">SKIN SHOP</span>
            </button>
            <button id="btn-audio-toggle" class="bg-slate-800
hover:bg-slate-700 text-slate-300 p-2 rounded-lg text-xs transition
border border-slate-700">
                <i class="fa-solid fa-volume-high" id="audio-icon"></i>
            </button>
        </div>
    </header>

    <!-- MAIN GAME CANVAS CONTAINER -->
    <main class="relative flex-1 w-full h-full overflow-hidden
bg-slate-950">
        <canvas id="gameCanvas" class="w-full h-full"></canvas>

        <div class="absolute top-4 right-4 flex flex-col space-y-2 z-10">
            <button id="btn-speed" class="glass-panel text-slate-200
```

```
hover:text-white px-3 py-1.5 rounded-lg text-xs font-bold transition flex items-center justify-center gap-1 border border-slate-700/80">
            <i class="fa-solid fa-forward"></i> Speed: <span id="speed-label" class="text-indigo-400">1x</span>
        </button>
        <button id="btn-clear-squad" class="glass-panel text-slate-300 hover:text-red-400 px-3 py-1.5 rounded-lg text-xs font-semibold transition border border-slate-700/80">
            <i class="fa-solid fa-trash-can"></i> Recall All
        </button>
    </div>

    <div id="deploy-hint" class="absolute bottom-4 left-1/2 -translate-x-1/2 pointer-events-none glass-panel px-4 py-1.5 rounded-full text-xs font-medium text-blue-300 border border-blue-500/30 transition duration-500 opacity-90 flex items-center gap-2">
        <i class="fa-solid fa-hand-pointer animate-bounce text-blue-400"></i> Tap/Click grid on the blue deployment zone to place selected troop
    </div>

    <div id="banner-announcement" class="absolute top-1/3 left-1/2 -translate-x-1/2 -translate-y-1/2 pointer-events-none transition-all duration-500 opacity-0 scale-90 z-30 text-center">
        <h2 id="banner-title" class="text-4xl md:text-6xl font-black tracking-widest text-transparent bg-clip-text bg-gradient-to-r from-amber-300 via-amber-400 to-orange-500
```

```
drop-shadow-[0_5px_15px_rgba(0,0,0,0.8)]">VICTORY!</h2>
            <p id="banner-sub" class="text-slate-200 font-semibold mt-2
text-sm md:text-base drop-shadow-md">+70 Coins & +1 Gem
Earned!</p>
        </div>
    </main>

    <!-- BOTTOM TOOLBAR -->
    <footer class="w-full glass-panel z-20 p-2 md:p-3 shadow-2xl flex
flex-col md:flex-row items-center justify-between gap-3 border-t
border-slate-800">
        <div class="flex items-center gap-2 overflow-x-auto w-full
md:w-auto pb-1 md:pb-0 scrollbar-none" id="troop-selector-container">
            <!-- Troop Cards Rendered via JS -->
        </div>

        <div class="flex items-center gap-3 w-full md:w-auto justify-end">
            <button id="btn-start-battle" class="w-full md:w-auto
bg-gradient-to-r from-emerald-500 via-green-500 to-teal-600
hover:from-emerald-400 hover:to-teal-500 text-slate-950 font-black
text-base px-8 py-3 rounded-xl shadow-lg shadow-emerald-500/20
transition hover:scale-105 active:scale-95 flex items-center justify-center
gap-2 tracking-wider">
                <i class="fa-solid fa-play text-lg"></i> START BATTLE
            </button>
        </div>
    </footer>
```

```
    <!-- MODAL: ITEM SKIN SHOP -->
    <div id="modal-shop" class="fixed inset-0 bg-slate-950/80
backdrop-blur-md z-50 flex items-center justify-center opacity-0
pointer-events-none transition-all duration-300">
        <div class="glass-panel w-11/12 max-w-4xl max-h-[85vh]
rounded-2xl p-6 border border-purple-500/30 flex flex-col shadow-2xl
relative">
            <div class="flex justify-between items-center pb-4 border-b
border-slate-800">
                <div class="flex items-center gap-3">
                    <div class="p-2.5 bg-purple-900/50 border
border-purple-500/40 rounded-xl text-purple-400">
                        <i class="fa-solid fa-shirt text-xl"></i>
                    </div>
                    <div>
                        <h2 class="text-xl font-bold text-slate-100">Cosmetic
Skin Shop</h2>
                        <p class="text-xs text-slate-400">Customize unit visuals,
aura glows, and particle effects with Gems</p>
                    </div>
                </div>
                <button id="btn-close-shop" class="text-slate-400
hover:text-white p-2 rounded-lg hover:bg-slate-800 transition">
                    <i class="fa-solid fa-xmark text-xl"></i>
                </button>
            </div>

            <div class="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-4
```

```
my-6 overflow-y-auto pr-1" id="shop-items-container">
            <!-- Shop items -->
        </div>

        <div class="pt-3 border-t border-slate-800 flex justify-between
items-center text-xs text-slate-400">
            <span>Earn <strong class="text-cyan-400"><i class="fa-solid
fa-gem"></i> 1 Gem</strong> for every victory round!</span>
            <div class="flex items-center gap-1 font-bold text-slate-200">
                Your Gems: <span id="shop-gem-count"
class="text-cyan-400">0</span>
            </div>
        </div>
    </div>
</div>

<!-- SCRIPT ENGINE -->
<script>
    /* --- AUDIO CONTROLLER --- */
    class AudioController {
        constructor() {
            this.ctx = null;
            this.muted = false;
        }

        init() {
            if (!this.ctx) {
                this.ctx = new (window.AudioContext ||
```

```
window.webkitAudioContext)();
            }
        }

        playTone(freq, type = 'sine', duration = 0.1, startVol = 0.1) {
            if (this.muted || !this.ctx) return;
            try {
                const osc = this.ctx.createOscillator();
                const gain = this.ctx.createGain();
                osc.type = type;
                osc.frequency.setValueAtTime(freq, this.ctx.currentTime);
                gain.gain.setValueAtTime(startVol, this.ctx.currentTime);
                gain.gain.exponentialRampToValueAtTime(0.0001,
this.ctx.currentTime + duration);
                osc.connect(gain);
                gain.connect(this.ctx.destination);
                osc.start();
                osc.stop(this.ctx.currentTime + duration);
            } catch(e){}
        }

        playSpawnSound() {
            this.init();
            this.playTone(320, 'triangle', 0.1, 0.15);
            setTimeout(() => this.playTone(480, 'triangle', 0.12, 0.15), 50);
        }

        playHitSound() {
```

```
            this.init();
            this.playTone(110, 'sawtooth', 0.08, 0.12);
        }

        playLaserSound() {
            this.init();
            if (this.muted || !this.ctx) return;
            try {
                const osc = this.ctx.createOscillator();
                const gain = this.ctx.createGain();
                osc.type = 'sawtooth';
                osc.frequency.setValueAtTime(900, this.ctx.currentTime);
                osc.frequency.exponentialRampToValueAtTime(120,
this.ctx.currentTime + 0.18);
                gain.gain.setValueAtTime(0.12, this.ctx.currentTime);
                gain.gain.linearRampToValueAtTime(0.01,
this.ctx.currentTime + 0.18);
                osc.connect(gain);
                gain.connect(this.ctx.destination);
                osc.start();
                osc.stop(this.ctx.currentTime + 0.18);
            } catch(e){}
        }

        playHealSound() {
            this.init();
            this.playTone(523.25, 'sine', 0.15, 0.1);
            setTimeout(() => this.playTone(659.25, 'sine', 0.2, 0.1), 100);
```

```
        }

        playCoinSound() {
            this.init();
            this.playTone(987.77, 'sine', 0.08, 0.15);
            setTimeout(() => this.playTone(1318.51, 'sine', 0.15, 0.15),
80);
        }

        playVictoryFanfare() {
            this.init();
            const notes = [523.25, 659.25, 783.99, 1046.50];
            notes.forEach((freq, idx) => {
                setTimeout(() => this.playTone(freq, 'triangle', 0.25, 0.2), idx
* 120);
            });
        }

        playDefeatSound() {
            this.init();
            const notes = [300, 260, 220, 180];
            notes.forEach((freq, idx) => {
                setTimeout(() => this.playTone(freq, 'sawtooth', 0.2, 0.2),
idx * 150);
            });
        }
    }
```

```
const audio = new AudioController();

/* --- GAME DATA DEFINITIONS --- */
const TROOP_TYPES = {
    KNIGHT: {
        id: 'KNIGHT',
        name: 'Vanguard Knight',
        role: 'Melee Tank',
        cost: 40,
        hp: 480,
        damage: 24,
        range: 42,
        speed: 1.2,
        attackSpeed: 1.1,
        icon: 'fa-shield-halved',
        color: '#3b82f6',
        description: 'Plate armor tank with heavy shield mitigation.',
        ability: 'Shield Block (-40% damage)'
    },
    PYRO: {
        id: 'PYRO',
        name: 'Arcane Pyromancer',
        role: 'AoE Mage',
        cost: 65,
        hp: 190,
        damage: 38,
        range: 180,
        speed: 1.0,
```

```
            attackSpeed: 1.6,
            icon: 'fa-fire-flame-curved',
            color: '#f97316',
            description: 'Hurls explosive fireballs dealing area splash
damage.',
            ability: 'Fire Splash AoE'
        },
        ASSASSIN: {
            id: 'ASSASSIN',
            name: 'Shadow Assassin',
            role: 'Flanker',
            cost: 55,
            hp: 210,
            damage: 52,
            range: 45,
            speed: 2.2,
            attackSpeed: 0.8,
            icon: 'fa-user-ninja',
            color: '#a855f7',
            description: 'Dual dagger stealth striker with shadowy speed.',
            ability: 'Shadow Blink & High Crit'
        },
        PRIESTESS: {
            id: 'PRIESTESS',
            name: 'Celestial Priestess',
            role: 'Healer Support',
            cost: 60,
            hp: 200,
```

```
            damage: 16,
            range: 160,
            speed: 1.1,
            attackSpeed: 1.8,
            icon: 'fa-wand-magic-sparkles',
            color: '#10b981',
            description: 'Floating halo priestess that radiates healing
pulses.',
            ability: 'Holy Healing Aura'
        },
        BEHEMOTH: {
            id: 'BEHEMOTH',
            name: 'Golem Behemoth',
            role: 'Heavy Brawler',
            cost: 100,
            hp: 950,
            damage: 58,
            range: 52,
            speed: 0.7,
            attackSpeed: 2.2,
            icon: 'fa-hand-fist',
            color: '#78350f',
            description: 'Colossal rocky giant with crystal core &
shockwave slams.',
            ability: 'Ground Slam Shockwave'
        },
        ARCHER: {
            id: 'ARCHER',
```

```
            name: 'Storm Archer',
            role: 'Rapid Marksman',
            cost: 50,
            hp: 200,
            damage: 22,
            range: 230,
            speed: 1.3,
            attackSpeed: 0.6,
            icon: 'fa-bow-arrow',
            color: '#eab308',
            description: 'Hooded ranger with a composite bow firing rapid volleys.',
            ability: 'Rapid Volley'
        },
        NECROMANCER: {
            id: 'NECROMANCER',
            name: 'Necromancer Lord',
            role: 'Summoner',
            cost: 85,
            hp: 250,
            damage: 26,
            range: 170,
            speed: 0.9,
            attackSpeed: 1.5,
            icon: 'fa-skull',
            color: '#6366f1',
            description: 'Bone staff wielder that summons Skeleton Minions.',
```

```
            ability: 'Skeleton Minion Summon'
        },
        VOID_MECH: {
            id: 'VOID_MECH',
            name: 'Void Mech',
            role: 'Laser Siege',
            cost: 110,
            hp: 400,
            damage: 75,
            range: 260,
            speed: 0.8,
            attackSpeed: 2.5,
            icon: 'fa-robot',
            color: '#06b6d4',
            description: 'Robotic chassis with dual cannons firing concentrated lasers.',
            ability: 'Piercing Laser Cannon'
        }
    };

    const SCENERIES = [
        { name: "Grassy Meadow", bgGradient: ['#0f2b1d', '#05140d'], accent: '#22c55e', gridLine: 'rgba(34, 197, 94, 0.12)', particles: 'leaf', decor: 'tree' },
        { name: "Desert Ruins", bgGradient: ['#2e1c0c', '#140b04'], accent: '#f59e0b', gridLine: 'rgba(245, 158, 11, 0.12)', particles: 'sand', decor: 'ruins' },
        { name: "Frozen Tundra", bgGradient: ['#0c2333', '#040d14'],
```

```
accent: '#38bdf8', gridLine: 'rgba(56, 189, 248, 0.12)', particles: 'snow', decor: 'ice' },
        { name: "Volcanic Wasteland", bgGradient: ['#2a0909', '#120202'], accent: '#ef4444', gridLine: 'rgba(239, 68, 68, 0.15)', particles: 'ember', decor: 'lava' },
        { name: "Cyberpunk Synthwave", bgGradient: ['#1a092b', '#0a0312'], accent: '#f43f5e', gridLine: 'rgba(244, 63, 94, 0.18)', particles: 'cyber', decor: 'cyber' },
        { name: "Mystic Mushroom Realm", bgGradient: ['#1e1035', '#0b0517'], accent: '#d946ef', gridLine: 'rgba(217, 70, 239, 0.15)', particles: 'spore', decor: 'shroom' },
        { name: "Celestial Galaxy", bgGradient: ['#090d2a', '#020412'], accent: '#818cf8', gridLine: 'rgba(129, 140, 248, 0.18)', particles: 'star', decor: 'crystal' }
    ];

    const SKINS = [
        { id: 'DEFAULT', name: 'Default Classic', cost: 0, glow: null, trail: null, desc: 'Standard tactical equipment' },
        { id: 'NEON', name: 'Neon Cyber Glow', cost: 3, glow: '#06b6d4', trail: '#38bdf8', desc: 'Futuristic cyan cybernetic outline' },
        { id: 'GOLDEN', name: 'Golden Royale', cost: 5, glow: '#fbbf24', trail: '#f59e0b', desc: 'Regal crown and radiant gold shine' },
        { id: 'VOID', name: 'Dark Void Shadow', cost: 6, glow: '#a855f7', trail: '#7e22ce', desc: 'Eerie dark void purple shadow clouds' },
        { id: 'FLAME', name: 'Flame Emperor', cost: 8, glow: '#f97316', trail: '#ef4444', desc: 'Fiery blazing trails and flame aura' },
        { id: 'CRYSTAL', name: 'Crystal Frost', cost: 10, glow: '#e0f2fe',
```

```
trail: '#a5f3fc', desc: 'Sparkling ice crystal shimmer effects' }
        ];

        /* --- GAME ENGINE CLASS --- */
        class GameEngine {
            constructor() {
                this.canvas = document.getElementById('gameCanvas');
                this.ctx = this.canvas.getContext('2d');

                this.level = 1;
                this.coins = 100;
                this.gems = 0;
                this.speedMultiplier = 1;
                this.gameState = 'PREPARATION';

                this.selectedTroopId = 'KNIGHT';
                this.equippedSkin = 'DEFAULT';
                this.unlockedSkins = ['DEFAULT'];

                this.playerBaseMaxHp = 1000;
                this.playerBaseHp = 1000;
                this.enemyBaseMaxHp = 1000;
                this.enemyBaseHp = 1000;

                this.playerUnits = [];
                this.battleUnits = [];
                this.projectiles = [];
                this.particles = [];
```

```
        this.floatingTexts = [];
        this.sceneryParticles = [];
        this.mapDecorations = [];

        this.gridCols = 12;
        this.gridRows = 6;
        this.deployMaxCol = 4;

        this.lastTime = performance.now();

        this.initResize();
        this.initEvents();
        this.initUI();
        this.initSceneryProps();

        requestAnimationFrame(this.loop.bind(this));
    }

    get currentScenery() {
        const idx = Math.floor((this.level - 1) / 10) %
SCENERIES.length;
        return SCENERIES[idx];
    }

    initResize() {
        const resize = () => {
            const rect =
this.canvas.parentElement.getBoundingClientRect();
```

```
        this.canvas.width = rect.width;
        this.canvas.height = rect.height;
        this.initSceneryProps();
    };
    window.addEventListener('resize', resize);
    resize();
}

initSceneryProps() {
    this.sceneryParticles = [];
    for (let i = 0; i < 45; i++) {
        this.sceneryParticles.push({
            x: Math.random() * this.canvas.width,
            y: Math.random() * this.canvas.height,
            radius: Math.random() * 2.5 + 1,
            speedX: (Math.random() - 0.5) * 0.5,
            speedY: Math.random() * 0.6 + 0.2,
            alpha: Math.random() * 0.6 + 0.2
        });
    }

    this.mapDecorations = [];
    for (let i = 0; i < 14; i++) {
        this.mapDecorations.push({
            x: Math.random() * this.canvas.width,
            y: Math.random() * this.canvas.height,
            size: Math.random() * 12 + 10,
            rot: Math.random() * Math.PI * 2
```

```
            });
        }
    }

    initUI() {
        this.updateHUD();
        this.renderTroopSelector();
        this.renderShopItems();
    }

    updateHUD() {
        document.getElementById('hud-level').innerText = this.level;
        document.getElementById('hud-scenery').innerText =
this.currentScenery.name;
        document.getElementById('hud-coins').innerText = this.coins;
        document.getElementById('hud-gems').innerText = this.gems;
        document.getElementById('shop-gem-count').innerText =
this.gems;

        const pPct = Math.max(0, (this.playerBaseHp /
this.playerBaseMaxHp) * 100);
        const ePct = Math.max(0, (this.enemyBaseHp /
this.enemyBaseMaxHp) * 100);

        document.getElementById('player-hp-bar').style.width =
`${pPct}%`;
        document.getElementById('player-hp-text').innerText =
`${Math.ceil(this.playerBaseHp)}/${this.playerBaseMaxHp}`;
```

```
            document.getElementById('enemy-hp-bar').style.width =
`${ePct}%`;
            document.getElementById('enemy-hp-text').innerText =
`${Math.ceil(this.enemyBaseHp)}/${this.enemyBaseMaxHp}`;
        }

        renderTroopSelector() {
            const container =
document.getElementById('troop-selector-container');
            container.innerHTML = '';

            Object.values(TROOP_TYPES).forEach(troop => {
                const isSelected = troop.id === this.selectedTroopId;
                const card = document.createElement('div');
                card.className = `glass-panel-interactive cursor-pointer
p-2 rounded-xl flex items-center gap-2.5 min-w-[150px]
sm:min-w-[170px] select-none ${isSelected ? 'ring-2 ring-blue-400
bg-blue-950/40' : ''}`;

                card.innerHTML = `
                    <div class="w-9 h-9 rounded-lg flex items-center
justify-center text-lg font-bold shadow-md" style="background-color:
${troop.color}25; color: ${troop.color}; border: 1px solid
${troop.color}60">
                        <i class="fa-solid ${troop.icon}"></i>
                    </div>
                    <div class="flex-1 min-w-0">
```

```
                    <div class="flex justify-between items-center">
                        <h4 class="text-xs font-bold text-slate-100
truncate">${troop.name}</h4>
                    </div>
                    <div class="flex justify-between items-center mt-1">
                        <span class="text-[10px] text-slate-400
font-medium">${troop.role}</span>
                        <span class="text-xs font-black text-amber-400 flex
items-center gap-0.5"><i class="fa-solid fa-coins text-[10px]"></i>
${troop.cost}</span>
                    </div>
                </div>
            `;

            card.onclick = () => {
                this.selectedTroopId = troop.id;
                this.renderTroopSelector();
            };

            container.appendChild(card);
        });
    }

    renderShopItems() {
        const container =
document.getElementById('shop-items-container');
        container.innerHTML = '';
```

```
        SKINS.forEach(skin => {
            const isUnlocked = this.unlockedSkins.includes(skin.id);
            const isEquipped = this.equippedSkin === skin.id;

            const card = document.createElement('div');
            card.className = `glass-panel p-4 rounded-xl border flex flex-col justify-between space-y-3 ${isEquipped ? 'border-purple-400 bg-purple-950/20' : 'border-slate-800'}`;

            card.innerHTML = `
                <div>
                    <div class="flex justify-between items-start">
                        <h3 class="font-bold text-sm text-slate-100">${skin.name}</h3>
                        ${isEquipped ? '<span class="bg-purple-500/20 border border-purple-400 text-purple-300 text-[10px] px-2 py-0.5 rounded-full font-bold">EQUIPPED</span>' : ''}
                    </div>
                    <p class="text-xs text-slate-400 mt-1">${skin.desc}</p>
                </div>
                <div class="pt-2 border-t border-slate-800/80 flex items-center justify-between">
                    ${isUnlocked ? `
                        <button class="btn-skin-action w-full ${isEquipped ? 'bg-slate-800 text-slate-400 cursor-default' : 'bg-purple-600 hover:bg-purple-500 text-white'} py-1.5 rounded-lg text-xs font-bold transition">
```

```
                        ${isEquipped ? 'Equipped' : 'Equip Skin'}
                    </button>
                `:`
                    <button class="btn-skin-action w-full bg-cyan-600
hover:bg-cyan-500 text-white py-1.5 rounded-lg text-xs font-bold
transition flex items-center justify-center gap-1.5">
                        <i class="fa-solid fa-gem"></i> Buy for
${skin.cost}
                    </button>
                `}
            </div>
        `;

        const btn = card.querySelector('.btn-skin-action');
        btn.onclick = () => {
            if (isUnlocked) {
                this.equippedSkin = skin.id;
                this.renderShopItems();
            } else {
                if (this.gems >= skin.cost) {
                    this.gems -= skin.cost;
                    this.unlockedSkins.push(skin.id);
                    this.equippedSkin = skin.id;
                    audio.playCoinSound();
                    this.updateHUD();
                    this.renderShopItems();
                } else {
                    this.showBanner("NOT ENOUGH GEMS!", "Win
```

```
rounds to earn more gems", "text-red-400");
                    }
                }
            };

            container.appendChild(card);
        });
    }

    initEvents() {
        this.canvas.addEventListener('click', (e) => {
            if (this.gameState !== 'PREPARATION') return;

            const rect = this.canvas.getBoundingClientRect();
            const x = e.clientX - rect.left;
            const y = e.clientY - rect.top;

            const grid = this.getGridCoords(x, y);
            if (grid.col < this.deployMaxCol) {
                this.toggleDeployUnit(grid.col, grid.row);
            }
        });

        document.getElementById('btn-start-battle').onclick = () => {
            if (this.gameState === 'PREPARATION') {
                if (this.playerUnits.length === 0) {
                    this.showBanner("NO TROOPS DEPLOYED!", "Click
the blue deployment zone to place units", "text-amber-400");
```

```
                return;
            }
            this.startBattlePhase();
        }
    };

    document.getElementById('btn-clear-squad').onclick = () => {
        if (this.gameState === 'PREPARATION') {
            this.playerUnits.forEach(u => this.coins += u.cost);
            this.playerUnits = [];
            audio.playCoinSound();
            this.updateHUD();
        }
    };

    document.getElementById('btn-speed').onclick = () => {
        if (this.speedMultiplier === 1) this.speedMultiplier = 2;
        else if (this.speedMultiplier === 2) this.speedMultiplier = 3;
        else this.speedMultiplier = 1;
        document.getElementById('speed-label').innerText =
`${this.speedMultiplier}x`;
    };

    const modalShop = document.getElementById('modal-shop');
    document.getElementById('btn-open-shop').onclick = () => {
        modalShop.classList.remove('opacity-0',
'pointer-events-none');
    };
```

```
            document.getElementById('btn-close-shop').onclick = () => {
                modalShop.classList.add('opacity-0', 'pointer-events-none');
            };

            document.getElementById('btn-audio-toggle').onclick = () => {
                audio.muted = !audio.muted;
                document.getElementById('audio-icon').className =
audio.muted ? 'fa-solid fa-volume-xmark text-red-400' : 'fa-solid
fa-volume-high';
            };
        }

        getGridCoords(x, y) {
            const cellW = this.canvas.width / this.gridCols;
            const cellH = this.canvas.height / this.gridRows;
            return {
                col: Math.floor(x / cellW),
                row: Math.floor(y / cellH),
                cellW, cellH
            };
        }

        toggleDeployUnit(col, row) {
            const existingIdx = this.playerUnits.findIndex(u => u.gridCol
=== col && u.gridRow === row);

            if (existingIdx >= 0) {
                this.coins += this.playerUnits[existingIdx].cost;
```

```
                this.playerUnits.splice(existingIdx, 1);
                audio.playHitSound();
            } else {
                const template = TROOP_TYPES[this.selectedTroopId];
                if (this.coins >= template.cost) {
                    this.coins -= template.cost;
                    this.playerUnits.push({
                        ...template,
                        gridCol: col,
                        gridRow: row
                    });
                    audio.playSpawnSound();
                } else {
                    this.showBanner("INSUFFICIENT COINS!", "Need more coins to purchase unit", "text-red-400");
                }
            }
            this.updateHUD();
        }

        startBattlePhase() {
            this.gameState = 'BATTLE';
            document.getElementById('btn-start-battle').disabled = true;

document.getElementById('btn-start-battle').classList.add('opacity-50', 'cursor-not-allowed');

            const cellW = this.canvas.width / this.gridCols;
```

```
            const cellH = this.canvas.height / this.gridRows;

            this.battleUnits = this.playerUnits.map(u => new Unit(
                u,
                cellW * u.gridCol + cellW / 2,
                cellH * u.gridRow + cellH / 2,
                'player'
            ));

            const enemyBudget = 60 + (this.level * 35);
            let remainingBudget = enemyBudget;
            const availableTypes = Object.keys(TROOP_TYPES);

            while (remainingBudget >= 40) {
                const randomTypeKey =
availableTypes[Math.floor(Math.random() * availableTypes.length)];
                const template = TROOP_TYPES[randomTypeKey];

                if (template.cost <= remainingBudget) {
                    const randomCol = Math.floor(Math.random() * 4) +
(this.gridCols - 4);
                    const randomRow = Math.floor(Math.random() *
this.gridRows);

                    this.battleUnits.push(new Unit(
                        template,
                        cellW * randomCol + cellW / 2,
                        cellH * randomRow + cellH / 2,
```

```
                    'enemy',
                    this.level
                ));
                remainingBudget -= template.cost;
            } else {
                break;
            }
        }

        this.projectiles = [];
        this.particles = [];
        this.floatingTexts = [];
    }

    loop(timestamp) {
        const dt = Math.min((timestamp - this.lastTime) / 1000, 0.1) *
this.speedMultiplier;
        this.lastTime = timestamp;

        this.update(dt);
        this.draw();

        requestAnimationFrame(this.loop.bind(this));
    }

    update(dt) {
        if (this.gameState === 'BATTLE') {
            this.battleUnits.forEach(unit => unit.update(dt, this));
```

```
            this.battleUnits = this.battleUnits.filter(unit => unit.hp > 0);

            const playersAlive = this.battleUnits.some(u => u.team ===
'player');
            const enemiesAlive = this.battleUnits.some(u => u.team
=== 'enemy');

            if (!enemiesAlive || this.enemyBaseHp <= 0) {
                this.endRound(true);
            } else if (!playersAlive || this.playerBaseHp <= 0) {
                this.endRound(false);
            }
        }

        this.projectiles.forEach(p => p.update(dt, this));
        this.projectiles = this.projectiles.filter(p => !p.destroyed);

        this.particles.forEach(p => p.update(dt));
        this.particles = this.particles.filter(p => p.life > 0);

        this.floatingTexts.forEach(ft => ft.update(dt));
        this.floatingTexts = this.floatingTexts.filter(ft => ft.life > 0);

        this.sceneryParticles.forEach(p => {
            p.x += p.speedX;
            p.y += p.speedY;
            if (p.y > this.canvas.height) p.y = 0;
            if (p.x > this.canvas.width) p.x = 0;
```

```
            if (p.x < 0) p.x = this.canvas.width;
        });
    }

    endRound(isVictory) {
        this.gameState = 'PREPARATION';

        const btnStart = document.getElementById('btn-start-battle');
        btnStart.disabled = false;
        btnStart.classList.remove('opacity-50', 'cursor-not-allowed');

        let coinReward = 50;

        if (isVictory) {
            coinReward += 20;
            this.gems += 1;
            this.level += 1;
            audio.playVictoryFanfare();
            this.showBanner("VICTORY!", `+70 Coins & +1 Gem
Earned!`, "text-amber-400");
        } else {
            coinReward += 10;
            audio.playDefeatSound();
            this.showBanner("DEFEAT!", `+60 Coins Earned. Prepare
& Retry!`, "text-rose-500");
        }

        this.coins += coinReward;
```

```
            this.playerBaseHp = this.playerBaseMaxHp;
            this.enemyBaseHp = this.enemyBaseMaxHp;

            this.battleUnits = [];
            this.projectiles = [];

            this.initSceneryProps();
            this.updateHUD();
            this.renderShopItems();
        }

        showBanner(title, sub, titleClass = "text-amber-400") {
            const banner =
document.getElementById('banner-announcement');
            const titleEl = document.getElementById('banner-title');
            const subEl = document.getElementById('banner-sub');

            titleEl.innerText = title;
            titleEl.className = `text-4xl md:text-6xl font-black
tracking-widest ${titleClass}
drop-shadow-[0_5px_15px_rgba(0,0,0,0.8)]`;
            subEl.innerText = sub;

            banner.classList.remove('opacity-0', 'scale-90');
            banner.classList.add('opacity-100', 'scale-100');

            setTimeout(() => {
```

```
            banner.classList.remove('opacity-100', 'scale-100');
            banner.classList.add('opacity-0', 'scale-90');
        }, 3000);
    }

    draw() {
        const ctx = this.ctx;
        const width = this.canvas.width;
        const height = this.canvas.height;

        // 1. Scenery Map Gradient
        const scenery = this.currentScenery;
        const bgGrad = ctx.createLinearGradient(0, 0, width, height);
        bgGrad.addColorStop(0, scenery.bgGradient[0]);
        bgGrad.addColorStop(1, scenery.bgGradient[1]);
        ctx.fillStyle = bgGrad;
        ctx.fillRect(0, 0, width, height);

        // 2. Map Environment Decorations
        this.drawMapDecorations(ctx, scenery);

        // 3. Ambient Weather Particles
        ctx.fillStyle = scenery.accent;
        this.sceneryParticles.forEach(p => {
            ctx.globalAlpha = p.alpha;
            ctx.beginPath();
            ctx.arc(p.x, p.y, p.radius, 0, Math.PI * 2);
            ctx.fill();
```

```
        });
        ctx.globalAlpha = 1.0;

        // 4. Tactical Deployment Grid
        const cellW = width / this.gridCols;
        const cellH = height / this.gridRows;

        ctx.lineWidth = 1;
        for (let c = 0; c <= this.gridCols; c++) {
            ctx.strokeStyle = (c <= this.deployMaxCol &&
this.gameState === 'PREPARATION')
                ? 'rgba(59, 130, 246, 0.25)'
                : scenery.gridLine;
            ctx.beginPath();
            ctx.moveTo(c * cellW, 0);
            ctx.lineTo(c * cellW, height);
            ctx.stroke();
        }
        for (let r = 0; r <= this.gridRows; r++) {
            ctx.strokeStyle = scenery.gridLine;
            ctx.beginPath();
            ctx.moveTo(0, r * cellH);
            ctx.lineTo(width, r * cellH);
            ctx.stroke();
        }

        if (this.gameState === 'PREPARATION') {
            ctx.fillStyle = 'rgba(59, 130, 246, 0.05)';
```

```
                ctx.fillRect(0, 0, cellW * this.deployMaxCol, height);
            }

            // 5. Base Fortresses
            this.drawBases(cellW, cellH);

            // 6. Preparation Units (Ghost Placed)
            if (this.gameState === 'PREPARATION') {
                this.playerUnits.forEach(u => {
                    const x = cellW * u.gridCol + cellW / 2;
                    const y = cellH * u.gridRow + cellH / 2;
                    this.drawVectorCharacter(ctx, x, y, u.id, 'player', 1, 0,
false, this.equippedSkin);
                });
            }

            // 7. Live Battle Units
            this.battleUnits.forEach(unit => unit.draw(ctx, this));

            // 8. Projectiles
            this.projectiles.forEach(p => p.draw(ctx));

            // 9. Particles
            this.particles.forEach(p => p.draw(ctx));

            // 10. Floating Damage Numbers
            this.floatingTexts.forEach(ft => ft.draw(ctx));
        }
```

```
        drawMapDecorations(ctx, scenery) {
            ctx.fillStyle = scenery.accent;
            ctx.globalAlpha = 0.12;

            this.mapDecorations.forEach(d => {
                ctx.save();
                ctx.translate(d.x, d.y);
                ctx.rotate(d.rot);

                if (scenery.decor === 'tree') {
                    ctx.beginPath();
                    ctx.arc(0, 0, d.size, 0, Math.PI * 2);
                    ctx.fill();
                } else if (scenery.decor === 'ruins') {
                    ctx.fillRect(-d.size/2, -d.size/2, d.size, d.size);
                } else if (scenery.decor === 'ice' || scenery.decor ===
'crystal') {
                    ctx.beginPath();
                    ctx.moveTo(0, -d.size);
                    ctx.lineTo(d.size/2, d.size);
                    ctx.lineTo(-d.size/2, d.size);
                    ctx.closePath();
                    ctx.fill();
                } else {
                    ctx.beginPath();
                    ctx.arc(0, 0, d.size * 0.8, 0, Math.PI * 2);
                    ctx.fill();
```

```
        }

        ctx.restore();
    });

    ctx.globalAlpha = 1.0;
}

drawBases(cellW, cellH) {
    const ctx = this.ctx;

    // Player Fortress (Left)
    ctx.save();
    ctx.translate(15, cellH * 1.5);
    ctx.fillStyle = '#1e3a8a';
    ctx.strokeStyle = '#3b82f6';
    ctx.lineWidth = 3;
    ctx.beginPath();
    ctx.roundRect(0, 0, 30, cellH * 3, 8);
    ctx.fill();
    ctx.stroke();

    ctx.fillStyle = '#60a5fa';
    ctx.fillRect(8, 15, 14, 14);
    ctx.fillRect(8, cellH * 1.5 - 7, 14, 14);
    ctx.fillRect(8, cellH * 3 - 29, 14, 14);
    ctx.restore();
```

```
            // Enemy Citadel (Right)
            ctx.save();
            ctx.translate(this.canvas.width - 45, cellH * 1.5);
            ctx.fillStyle = '#881337';
            ctx.strokeStyle = '#f43f5e';
            ctx.lineWidth = 3;
            ctx.beginPath();
            ctx.roundRect(0, 0, 30, cellH * 3, 8);
            ctx.fill();
            ctx.stroke();

            ctx.fillStyle = '#f87171';
            ctx.fillRect(8, 15, 14, 14);
            ctx.fillRect(8, cellH * 1.5 - 7, 14, 14);
            ctx.fillRect(8, cellH * 3 - 29, 14, 14);
            ctx.restore();
        }

        /* --- PROCEDURAL VECTOR GRAPHICS ENGINE --- */
        drawVectorCharacter(ctx, x, y, typeId, team, scale = 1,
walkCycle = 0, isAttacking = false, skinId = 'DEFAULT') {
            ctx.save();
            ctx.translate(x, y);

            const skin = SKINS.find(s => s.id === skinId) || SKINS[0];

            // Cosmetic Skin Aura Glow
            if (skin.glow) {
```

```
            ctx.shadowColor = skin.glow;
            ctx.shadowBlur = 12;

            if (skinId === 'GOLDEN') {
                ctx.fillStyle = '#fbbf24';
                ctx.beginPath();
                ctx.arc(0, -22, 5, 0, Math.PI * 2);
                ctx.fill();
            }
        }

        // Facing Direction Flip
        if (team === 'enemy') ctx.scale(-1, 1);

        const isPlayer = team === 'player';
        const mainColor = isPlayer ? TROOP_TYPES[typeId]?.color ||
'#3b82f6' : '#ef4444';
        const bobY = Math.sin(walkCycle * Math.PI * 2) * 2;

        ctx.translate(0, bobY);

        /* DRAW CHARACTER PROCEDURAL GRAPHICS
ACCORDING TO TYPE */
        switch (typeId) {
            case 'KNIGHT':
                // Armor Body
                ctx.fillStyle = '#64748b';
                ctx.fillRect(-10, -12, 20, 20);
```

```
ctx.fillStyle = mainColor;
ctx.fillRect(-8, -10, 16, 16);

// Helmet & Visor
ctx.fillStyle = '#475569';
ctx.beginPath();
ctx.arc(0, -16, 10, 0, Math.PI * 2);
ctx.fill();
ctx.fillStyle = '#0f172a';
ctx.fillRect(1, -18, 7, 4);

// Shield
ctx.fillStyle = mainColor;
ctx.strokeStyle = '#f8fafc';
ctx.lineWidth = 2;
ctx.beginPath();
ctx.arc(-8, 0, 9, 0, Math.PI * 2);
ctx.fill();
ctx.stroke();

// Sword / Attack Swing
ctx.save();
ctx.translate(8, -2);
if (isAttacking) ctx.rotate(Math.PI / 3);
ctx.fillStyle = '#e2e8f0';
ctx.fillRect(0, -14, 4, 16);
ctx.fillStyle = '#94a3b8';
ctx.fillRect(-3, -2, 10, 3);
```

```
        ctx.restore();
        break;

    case 'PYRO':
        // Hooded Robe
        ctx.fillStyle = '#c2410c';
        ctx.beginPath();
        ctx.moveTo(0, -22);
        ctx.lineTo(-12, 10);
        ctx.lineTo(12, 10);
        ctx.closePath();
        ctx.fill();

        // Glowing Face/Eyes
        ctx.fillStyle = '#fbbf24';
        ctx.beginPath();
        ctx.arc(3, -14, 2, 0, Math.PI * 2);
        ctx.fill();

        // Fiery Staff
        ctx.fillStyle = '#78350f';
        ctx.fillRect(8, -18, 3, 26);
        ctx.fillStyle = '#f97316';
        ctx.beginPath();
        ctx.arc(9.5, -20, 5, 0, Math.PI * 2);
        ctx.fill();
        break;
```

```
case 'ASSASSIN':
    // Sleek Stealth Body
    ctx.fillStyle = '#3b0764';
    ctx.fillRect(-8, -12, 16, 18);

    // Mask / Hood
    ctx.fillStyle = '#581c87';
    ctx.beginPath();
    ctx.arc(0, -15, 8, 0, Math.PI * 2);
    ctx.fill();
    ctx.fillStyle = '#c084fc';
    ctx.fillRect(2, -16, 5, 2);

    // Dual Daggers
    ctx.fillStyle = '#cbd5e1';
    ctx.fillRect(8, -8, 8, 2);
    ctx.fillRect(-12, -4, 8, 2);
    break;

case 'PRIESTESS':
    // Floating Halo
    ctx.strokeStyle = '#fde047';
    ctx.lineWidth = 2;
    ctx.beginPath();
    ctx.ellipse(0, -24, 8, 3, 0, 0, Math.PI * 2);
    ctx.stroke();

    // Radiant Robes
```

```
        ctx.fillStyle = '#047857';
        ctx.beginPath();
        ctx.moveTo(0, -18);
        ctx.lineTo(-10, 10);
        ctx.lineTo(10, 10);
        ctx.closePath();
        ctx.fill();

        // Staff with Floating Orb
        ctx.fillStyle = '#d97706';
        ctx.fillRect(8, -16, 2, 24);
        ctx.fillStyle = '#34d399';
        ctx.beginPath();
        ctx.arc(9, -18, 4, 0, Math.PI * 2);
        ctx.fill();
        break;

    case 'BEHEMOTH':
        // Colossal Stone Body
        ctx.fillStyle = '#451a03';
        ctx.fillRect(-16, -18, 32, 26);

        // Glowing Crystal Core
        ctx.fillStyle = '#f59e0b';
        ctx.beginPath();
        ctx.moveTo(0, -12);
        ctx.lineTo(6, -4);
        ctx.lineTo(0, 4);
```

```
        ctx.lineTo(-6, -4);
        ctx.closePath();
        ctx.fill();

        // Massive Fists
        ctx.fillStyle = '#78350f';
        ctx.fillRect(12, -4, 10, 10);
        ctx.fillRect(-22, -4, 10, 10);
        break;

    case 'ARCHER':
        // Ranger Tunic
        ctx.fillStyle = '#15803d';
        ctx.fillRect(-8, -12, 16, 18);

        // Hood
        ctx.fillStyle = '#166534';
        ctx.beginPath();
        ctx.arc(0, -15, 8, 0, Math.PI * 2);
        ctx.fill();

        // Composite Bow
        ctx.strokeStyle = '#854d0e';
        ctx.lineWidth = 3;
        ctx.beginPath();
        ctx.arc(6, -6, 12, -Math.PI / 2, Math.PI / 2);
        ctx.stroke();
        break;
```

```
case 'NECROMANCER':
    // Bone Staff & Skull Mask
    ctx.fillStyle = '#312e81';
    ctx.fillRect(-9, -12, 18, 20);

    ctx.fillStyle = '#f8fafc';
    ctx.beginPath();
    ctx.arc(0, -15, 7, 0, Math.PI * 2);
    ctx.fill();
    ctx.fillStyle = '#020617';
    ctx.fillRect(-3, -16, 3, 3);
    ctx.fillRect(1, -16, 3, 3);

    // Staff
    ctx.fillStyle = '#475569';
    ctx.fillRect(8, -18, 3, 26);
    ctx.fillStyle = '#22c55e';
    ctx.beginPath();
    ctx.arc(9.5, -20, 4, 0, Math.PI * 2);
    ctx.fill();
    break;

case 'VOID_MECH':
    // Robotic Chassis
    ctx.fillStyle = '#155e75';
    ctx.fillRect(-12, -16, 24, 22);
```

```
                // Eye Sensor Visor
                ctx.fillStyle = '#06b6d4';
                ctx.fillRect(0, -12, 10, 4);

                // Dual Shoulder Cannons
                ctx.fillStyle = '#0e7490';
                ctx.fillRect(4, -22, 6, 10);
                ctx.fillRect(-10, -22, 6, 10);
                break;

            case 'SKELETON':
                // Skeleton Minion
                ctx.fillStyle = '#e2e8f0';
                ctx.fillRect(-6, -10, 12, 14);
                ctx.beginPath();
                ctx.arc(0, -12, 6, 0, Math.PI * 2);
                ctx.fill();
                ctx.fillStyle = '#0f172a';
                ctx.fillRect(1, -14, 2, 2);
                break;
        }

        ctx.restore();
    }
}

/* --- UNIT CLASS --- */
class Unit {
```

```
        constructor(template, x, y, team, levelBonus = 1) {
            Object.assign(this, template);
            this.x = x;
            this.y = y;
            this.team = team;

            const levelMult = team === 'enemy' ? (1 + (levelBonus - 1) *
0.12) : 1;
            this.maxHp = Math.floor(template.hp * levelMult);
            this.hp = this.maxHp;
            this.damage = Math.floor(template.damage * levelMult);

            this.attackCooldown = 0;
            this.target = null;
            this.summonTimer = 0;

            this.walkCycle = Math.random();
            this.isAttacking = false;
            this.attackAnimTimer = 0;
        }

        update(dt, engine) {
            if (this.hp <= 0) return;

            this.attackCooldown -= dt;
            this.summonTimer += dt;

            if (this.attackAnimTimer > 0) {
```

```
            this.attackAnimTimer -= dt;
            if (this.attackAnimTimer <= 0) this.isAttacking = false;
        }

        // Necromancer Summoning
        if (this.id === 'NECROMANCER' && this.summonTimer >= 6)
{
            this.summonTimer = 0;
            engine.battleUnits.push(new Unit(
                { id: 'SKELETON', name: 'Skeleton Minion', cost: 0, hp:
85, damage: 14, range: 35, speed: 1.5, attackSpeed: 1.0, color:
'#94a3b8' },
                this.x + (this.team === 'player' ? 20 : -20),
                this.y + (Math.random() * 30 - 15),
                this.team
            ));
            engine.particles.push(new Particle(this.x, this.y, '#a855f7',
15));
        }

        this.target = this.findTarget(engine);

        if (this.target) {
            const dist = Math.hypot(this.target.x - this.x, this.target.y -
this.y);

            if (dist <= this.range) {
                if (this.attackCooldown <= 0) {
```

```
                    this.attack(this.target, engine);
                    this.attackCooldown = this.attackSpeed;
                    this.isAttacking = true;
                    this.attackAnimTimer = 0.25;
                }
            } else {
                const angle = Math.atan2(this.target.y - this.y,
this.target.x - this.x);
                this.x += Math.cos(angle) * this.speed * 60 * dt;
                this.y += Math.sin(angle) * this.speed * 60 * dt;
                this.walkCycle = (this.walkCycle + dt * 4) % 1;
            }
        } else {
            const targetX = this.team === 'player' ?
engine.canvas.width - 30 : 30;
            const angle = Math.atan2(engine.canvas.height / 2 - this.y,
targetX - this.x);
            this.x += Math.cos(angle) * this.speed * 60 * dt;
            this.y += Math.sin(angle) * this.speed * 60 * dt;
            this.walkCycle = (this.walkCycle + dt * 4) % 1;

            if (Math.abs(this.x - targetX) < 40) {
                if (this.attackCooldown <= 0) {
                    if (this.team === 'player') engine.enemyBaseHp -=
this.damage;
                    else engine.playerBaseHp -= this.damage;

                    engine.floatingTexts.push(new FloatingText(targetX,
```

```
this.y, `-${this.damage}`, '#f43f5e'));
                    audio.playHitSound();
                    this.attackCooldown = this.attackSpeed;
                    this.isAttacking = true;
                    this.attackAnimTimer = 0.25;
                }
            }
        }
    }

    findTarget(engine) {
        let closest = null;
        let minDist = Infinity;

        engine.battleUnits.forEach(other => {
            if (other.team !== this.team && other.hp > 0) {
                const dist = Math.hypot(other.x - this.x, other.y - this.y);
                if (dist < minDist) {
                    minDist = dist;
                    closest = other;
                }
            }
        });

        return closest;
    }

    attack(target, engine) {
```

```
            if (this.id === 'PRIESTESS') {
                let lowestAlly = null;
                let lowestPct = 1.0;
                engine.battleUnits.forEach(ally => {
                    if (ally.team === this.team && ally.hp < ally.maxHp) {
                        const pct = ally.hp / ally.maxHp;
                        if (pct < lowestPct) {
                            lowestPct = pct;
                            lowestAlly = ally;
                        }
                    }
                });

                if (lowestAlly) {
                    const healAmt = 38;
                    lowestAlly.hp = Math.min(lowestAlly.maxHp,
lowestAlly.hp + healAmt);
                    engine.floatingTexts.push(new FloatingText(lowestAlly.x,
lowestAlly.y, `+${healAmt}`, '#22c55e'));
                    engine.particles.push(new Particle(lowestAlly.x,
lowestAlly.y, '#22c55e', 12));
                    audio.playHealSound();
                }
                return;
            }

            if (this.id === 'VOID_MECH') {
                audio.playLaserSound();
```

```
                engine.projectiles.push(new LaserBeam(this.x, this.y,
target.x, target.y, this.damage, this.team));
                return;
            }

            if (this.range > 60) {
                audio.playHitSound();
                engine.projectiles.push(new Projectile(this.x, this.y, target,
this.damage, this.color, this.id === 'PYRO'));
            } else {
                let finalDamage = this.damage;

                if (target.id === 'KNIGHT') {
                    finalDamage = Math.floor(finalDamage * 0.6);
                }

                target.hp -= finalDamage;
                engine.floatingTexts.push(new FloatingText(target.x,
target.y, `-${finalDamage}`, '#f87171'));
                engine.particles.push(new Particle(target.x, target.y,
'#ef4444', 8));
                audio.playHitSound();
            }
        }

        draw(ctx, engine) {
            engine.drawVectorCharacter(
                ctx,
```

```
            this.x,
            this.y,
            this.id,
            this.team,
            1,
            this.walkCycle,
            this.isAttacking,
            this.team === 'player' ? engine.equippedSkin : 'DEFAULT'
        );

        // Detailed Health Bar Overlay
        const hpPct = Math.max(0, this.hp / this.maxHp);
        ctx.fillStyle = 'rgba(15, 23, 42, 0.85)';
        ctx.fillRect(this.x - 18, this.y - 32, 36, 6);

        ctx.fillStyle = this.team === 'player' ? '#3b82f6' : '#ef4444';
        ctx.fillRect(this.x - 18, this.y - 32, 36 * hpPct, 6);

        ctx.strokeStyle = '#020617';
        ctx.lineWidth = 1;
        ctx.strokeRect(this.x - 18, this.y - 32, 36, 6);
    }
}

/* --- PROJECTILE CLASS --- */
class Projectile {
    constructor(x, y, target, damage, color, isSplash = false) {
        this.x = x;
```

```
        this.y = y;
        this.target = target;
        this.damage = damage;
        this.color = color;
        this.isSplash = isSplash;
        this.speed = 360;
        this.destroyed = false;
    }

    update(dt, engine) {
        if (!this.target || this.target.hp <= 0) {
            this.destroyed = true;
            return;
        }

        const angle = Math.atan2(this.target.y - this.y, this.target.x -
this.x);
        this.x += Math.cos(angle) * this.speed * dt;
        this.y += Math.sin(angle) * this.speed * dt;

        if (Math.hypot(this.target.x - this.x, this.target.y - this.y) < 15) {
            this.impact(engine);
            this.destroyed = true;
        }
    }

    impact(engine) {
        if (this.isSplash) {
```

```
                engine.battleUnits.forEach(unit => {
                    if (unit.team !== this.target.team && Math.hypot(unit.x - this.x, unit.y - this.y) < 70) {
                        unit.hp -= this.damage;
                        engine.floatingTexts.push(new FloatingText(unit.x, unit.y, `-${this.damage}`, '#f97316'));
                    }
                });
                engine.particles.push(new Particle(this.x, this.y, '#f97316', 22));
            } else {
                this.target.hp -= this.damage;
                engine.floatingTexts.push(new FloatingText(this.target.x, this.target.y, `-${this.damage}`, '#f87171'));
                engine.particles.push(new Particle(this.x, this.y, this.color, 8));
            }
        }

        draw(ctx) {
            ctx.fillStyle = this.color;
            ctx.beginPath();
            ctx.arc(this.x, this.y, 6, 0, Math.PI * 2);
            ctx.fill();
        }
    }

    /* --- LASER BEAM EFFECT --- */
```

```
class LaserBeam {
    constructor(x1, y1, x2, y2, damage, team) {
        this.x1 = x1;
        this.y1 = y1;
        this.x2 = x2;
        this.y2 = y2;
        this.damage = damage;
        this.team = team;
        this.life = 0.22;
        this.destroyed = false;
    }

    update(dt) {
        this.life -= dt;
        if (this.life <= 0) this.destroyed = true;
    }

    draw(ctx) {
        ctx.strokeStyle = '#06b6d4';
        ctx.lineWidth = 5;
        ctx.beginPath();
        ctx.moveTo(this.x1, this.y1);
        ctx.lineTo(this.x2, this.y2);
        ctx.stroke();

        ctx.strokeStyle = '#ffffff';
        ctx.lineWidth = 2;
        ctx.beginPath();
```

```
        ctx.moveTo(this.x1, this.y1);
        ctx.lineTo(this.x2, this.y2);
        ctx.stroke();
    }
}

/* --- PARTICLE & FLOATING TEXT VISUALS --- */
class Particle {
    constructor(x, y, color, count = 10) {
        this.particles = [];
        for (let i = 0; i < count; i++) {
            this.particles.push({
                x, y,
                vx: (Math.random() - 0.5) * 140,
                vy: (Math.random() - 0.5) * 140,
                radius: Math.random() * 3.5 + 1,
                color,
                alpha: 1.0
            });
        }
        this.life = 0.45;
    }

    update(dt) {
        this.life -= dt;
        this.particles.forEach(p => {
            p.x += p.vx * dt;
            p.y += p.vy * dt;
```

```
            p.alpha = Math.max(0, this.life / 0.45);
        });
    }

    draw(ctx) {
        this.particles.forEach(p => {
            ctx.globalAlpha = p.alpha;
            ctx.fillStyle = p.color;
            ctx.beginPath();
            ctx.arc(p.x, p.y, p.radius, 0, Math.PI * 2);
            ctx.fill();
        });
        ctx.globalAlpha = 1.0;
    }
}

class FloatingText {
    constructor(x, y, text, color) {
        this.x = x;
        this.y = y;
        this.text = text;
        this.color = color;
        this.life = 0.85;
    }

    update(dt) {
        this.life -= dt;
        this.y -= 22 * dt;
```

```
        }

        draw(ctx) {
            ctx.globalAlpha = Math.max(0, this.life / 0.85);
            ctx.font = '900 15px Outfit';
            ctx.fillStyle = this.color;
            ctx.fillText(this.text, this.x - 12, this.y);
            ctx.globalAlpha = 1.0;
        }
    }

    // Launch Game on Load
    window.onload = () => {
        window.game = new GameEngine();
    };
  </script>
</body>
</html>
```